

SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Objectives

01 What is a class diagram

02 Class box syntax, static and abstract notation

03 Visibility notation

04 Inheritance & realization

05 Association, aggregation & composition

06 Reading and creating diagrams

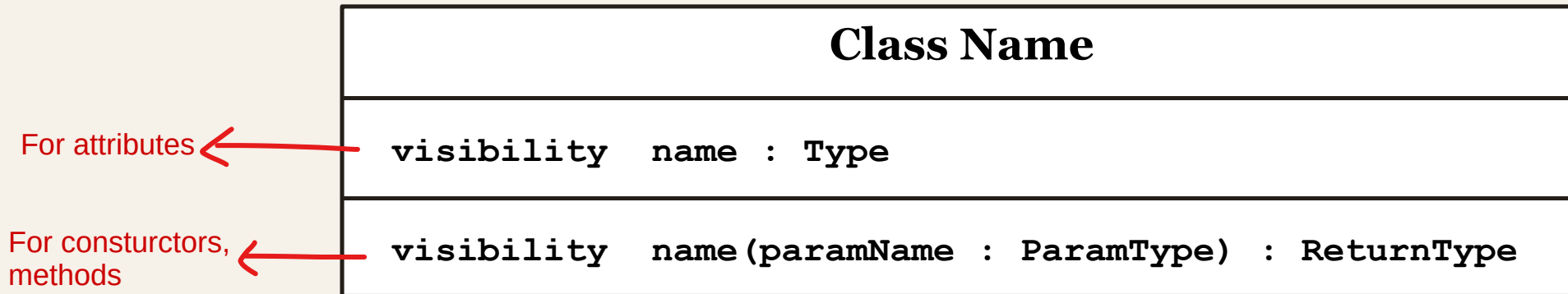
✓ What is a UML Class Diagram

- ✓ • UML stands for **Unified Modeling Language**
- ✓ • It can be used to model an **object-oriented system**
- ✓ • A UML class diagram depicts the structure of a system by showing its classes, their attributes and methods, and the relationships between them
- ✓ • In plain terms, it is a blueprint of the classes in a program, without any of the actual code

✓ Why Draw It First

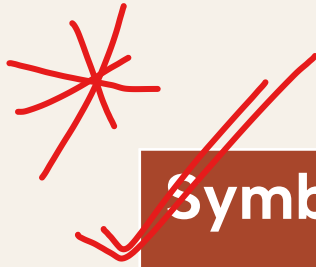
- ✓ ● It's easier to spot a bad design on a whiteboard than after writing 200 lines of code
- ✓ ● It's a shared visual language that is understood by non-technical people

Class Box Syntax



- ✓ • Top: the class name. Middle: attributes. Bottom: methods
- ✓ • Attributes: visibility symbol, then name, a colon, then the type
- ✓ • Methods: visibility symbol, name, parameters in brackets (name: type), a colon, then the return type

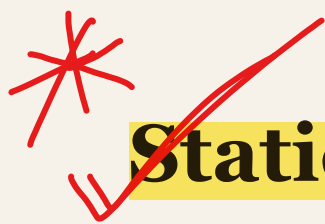
Visibility Notation



Symbol	Meaning
+	public
-	private
#	protected
~	package-private

A Filled Example

Student
- name : String - id : int
+ Student(name : String, id : int) + getName() : String + registerCourse(courseName : String) : void



Static Members: Shown Underlined

- totalCount : int ← *a static attribute*

+ getTotalCount () : int ← *a static method*

- ✓ • A static field or method belongs to the class itself, not to any one object
- ✓ • UML shows this with an underline

✓ Abstract Members: Shown in Italics

Animal ← *an abstract class*

+ *eat()* : *void* ← *an abstract method*

- ✓ • An abstract class can never be instantiated directly, only its concrete subclasses can
- ✓ • UML shows this by writing the class name in italics
- ✓ • An abstract method inside the box is written in italics the same way

A Filled Example

Employee

```
- name : String  
- id : int  
- totalEmployees : int = 0
```

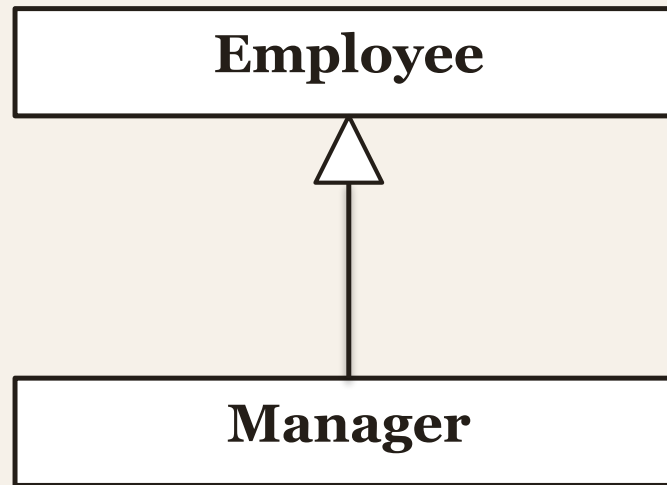
```
+ getName() : String  
+ calculatePay(baseSalary: double) : double  
+ getTotalEmployees() : int
```

A default value can be specified for attributes

✓ is-a Relationships

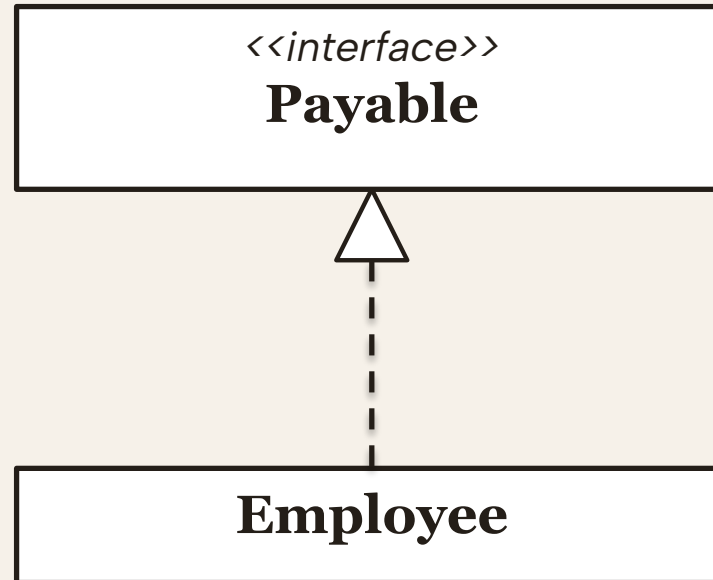
- ✓ ● We are already familiar with inheritance
- ✓ ● **Realization** is an '**implements**' relationship where a class fulfills a contract defined by an interface
- ✓ ● Both are drawn with a hollow triangle arrowhead pointing at the more general type
- ✓ ● The line style is the only thing that tells them apart

✓ Inheritance



- ✓ • Solid line + hollow triangle = extends
- ✓ • The arrow points **FROM** the subclass **TO** the superclass

✓ Realization (Implementing an Interface)



✓ • Dashed line + hollow triangle = implements

✓ • `<<interface>>` above the name marks it as an interface

extends -> Inheritance
implements -> Realization

✓ Three other Relationships

- ✓ ● Three relationships describe how one class can relate to another:
association, aggregation, and composition
- ✓ ● They look similar but mean very different things
- ✓ ● The key question for telling them apart: does one side own the other, and does one depend on the other to exist?

✓ Association

```
class Doctor {  
    private Patient currentPatient;  
  
    void diagnose(Patient patient) {  
        System.out.println("Diagnosing " + patient.getName());  
    }  
  
    void setCurrentPatient(Patient patient) {  
        this.currentPatient = patient;  
    }  
}
```

- ✓ Patient is stored as a field in Doctor
- ✓ Neither class is a container for the other, they're just related
- ✓ Both classes, and their objects, can exist completely independently

✓ Association: In UML



- ✓ • Any **general relationship** between two classes
- ✓ • Doctor does not own Patients/Doctor is not made up of patients
- ✓ • Depicted by a plain line
- ✓ • Works whether the reference is a stored field, or just used briefly like in `diagnose()`
- ✓ • Read simply as "Doctor is associated with Patient"

✓ Aggregation

```
class Team {  
    private List<Player> players;  
  
    void addPlayer(Player player) {  
        players.add(player);  
    }  
}  
  
class Player {  
    private String name;  
}
```

- ✓ • Team is a **container** for Players, this is a real **has-a** relationship
- ✓ • Players are added from outside, not created inside Team
- ✓ • Deleting a Team doesn't delete its Players, they can join another Team

✓ Aggregation: In UML



- ✓ A hollow diamond, sitting at the whole/container end – Team
- ✓ This is a has-a relationship, but the parts can **exist independently**
- ✓ Team 'contains' Player but it does not 'own' Player
- ✓ A Player can leave a Team and still exist, and still play in another team

Composition

```
class Car {  
    private Engine engine;  
  
    Car() {  
        engine = new Engine();  
    }  
}  
  
class Engine {  
    void start() { System.out.println("Engine started"); }  
}
```

- ✓ Car creates its own Engine **internally**
- ✓ No other code holds a separate reference to that Engine
- ✓ The Engine's lifetime is entirely tied to this one Car

✓ Composition: In UML



- ✓ • A filled diamond, sitting at the whole/container end — Car
- ✓ • This is a **has-a** relationship, and the parts **depend** on the whole
- ✓ • Car 'owns' Engine
- ✓ • Once this Car is gone, so is its Engine

* Association vs Aggregation vs Composition

Relationship	Ownership	Lifecycle	Notation
Association	No ownership	Fully independent	Plain line
Aggregation	Whole has parts	Parts can outlive the whole	Hollow diamond
Composition	Whole owns parts	Parts die with the whole	Filled diamond

- * ☒ Is one class genuinely ***made up*** of the other? If not, it's **Association**
- ☒ If yes: can the contained object exist **independently**, if the container is deleted?
- ☒ Survives independently → **Aggregation**. Does not survive independently → **Composition**

✓ What is Multiplicity

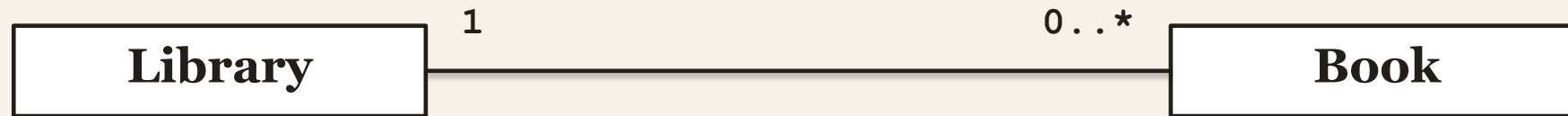
- ✓ • Multiplicity states how many instances of one class can relate to how many instances of another
- ✓ • Written as a number or range placed near each end of the relationship line
- ✓ • It applies to association, aggregation, and composition lines alike

Multiplicity Notations



Notation	Meaning	Example
1	Exactly one	Every Car has exactly 1 Engine
0..1	Zero or one (optional)	A Person has 0..1 Passport
0..* (or *)	Zero or many	A Playlist has 0..* Songs
1..*	One or many (at least one)	A Team has 1..* Players
5	An exact number	A Pentagon has exactly 5 Sides
2..4	A specific range	A Car has 2..4 Doors

✓ Multiplicity in a Diagram



- ✓ Read from the Book end: "a Library has 0 or many Books"
- ✓ Read from the Library end: "a Book belongs to exactly 1 Library"

✓ Common mistakes

- ✓ ● Drawing the inheritance arrow pointing from parent down to child
- ✓ ● Putting the diamond at the “part” end instead of the “whole” end
- ✓ ● The arrow or diamond always sits next to the more general, or more “whole” side
- ✓ ● Mixing up multiplicity notations

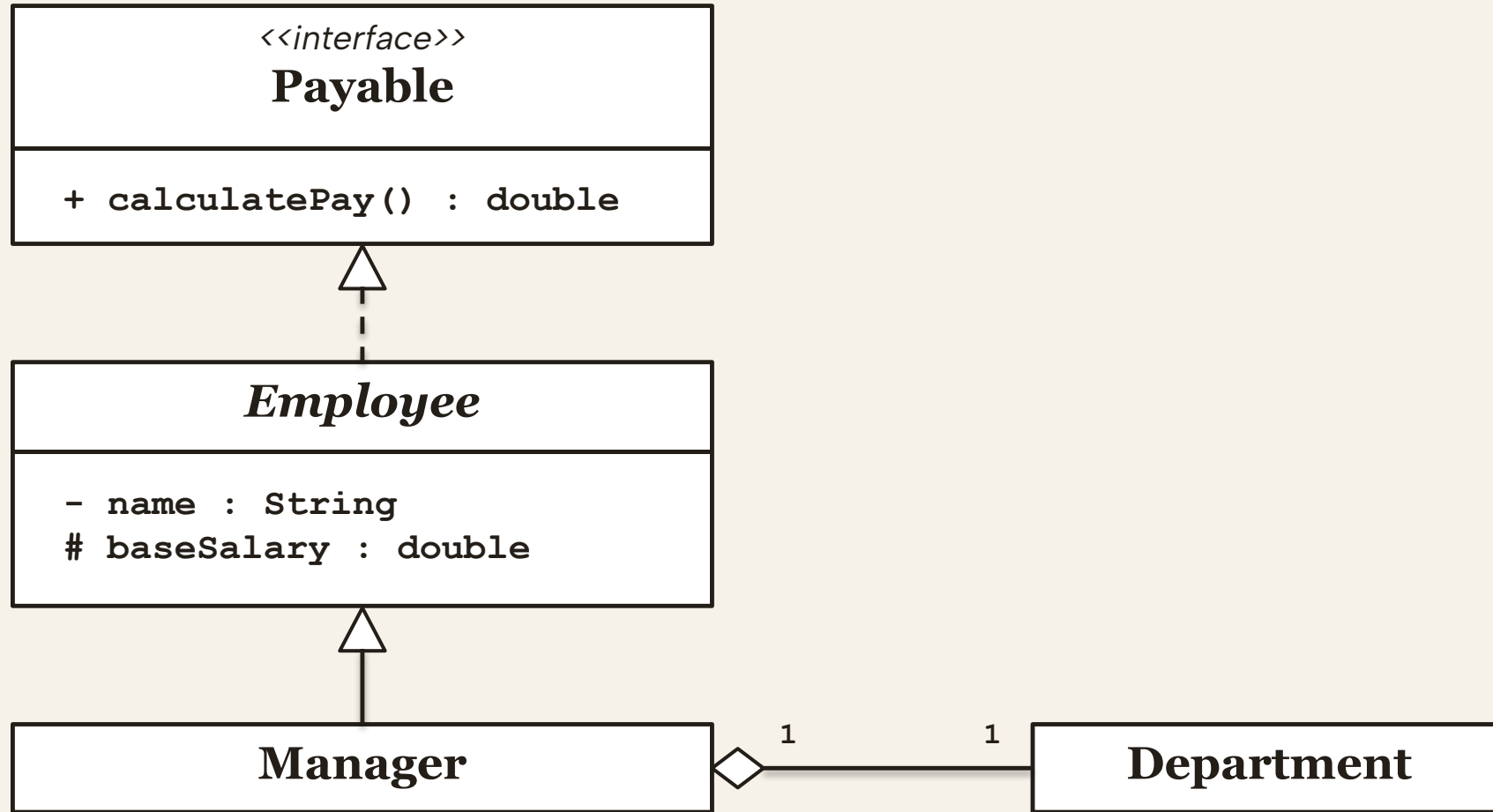
✓ Comprehensive Example 1: Code to Diagram

- ✓ ● Given a set of Java classes, we'll draw the matching class diagram
- ✓ ● Work through each class first, then each relationship between them

The Code

```
interface Payable {  
    double calculatePay();  
}  
  
abstract class Employee implements Payable {  
    private String name;  
    protected double baseSalary;  
}  
  
class Manager extends Employee {  
    private Department department;  
    Manager(Department department) { this.department = department; }  
}
```

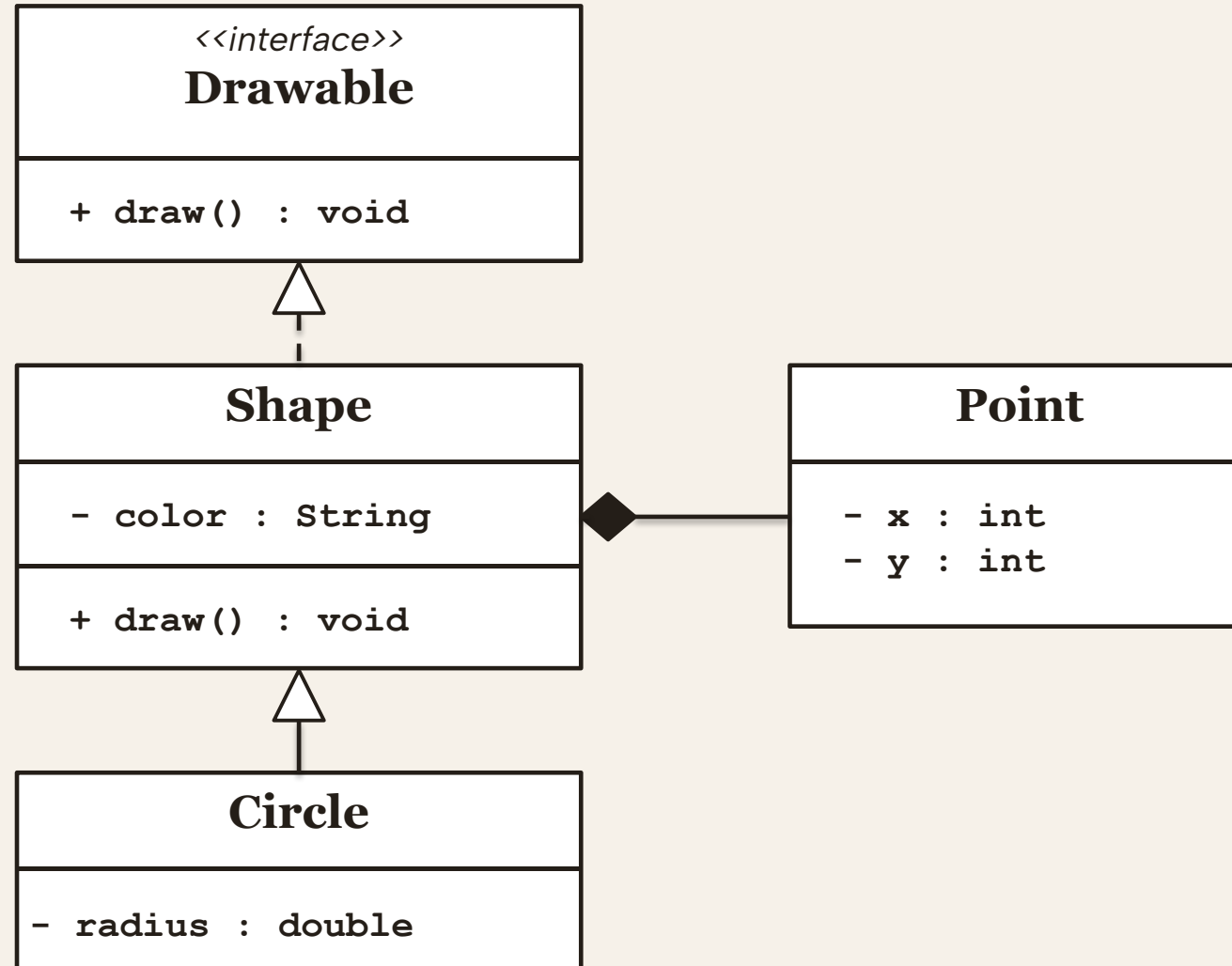
The Resulting Diagram



✓ Comprehensive Example 2: Diagram to Code

- ✓ • This time we start from a diagram, and write the Java classes it describes
- ✓ • Read each box first, then translate each relationship into the right keyword

The Diagram



The Resulting Code

```
interface Drawable {  
    void draw();  
}  
class Shape implements Drawable {  
    private String color;  
    private Point center;  
    public Shape() { center = new Point(); }  
    public void draw() { }  
}  
class Circle extends Shape {  
    private double radius;  
}  
class Point {  
    private int x;  
    private int y;  
}
```

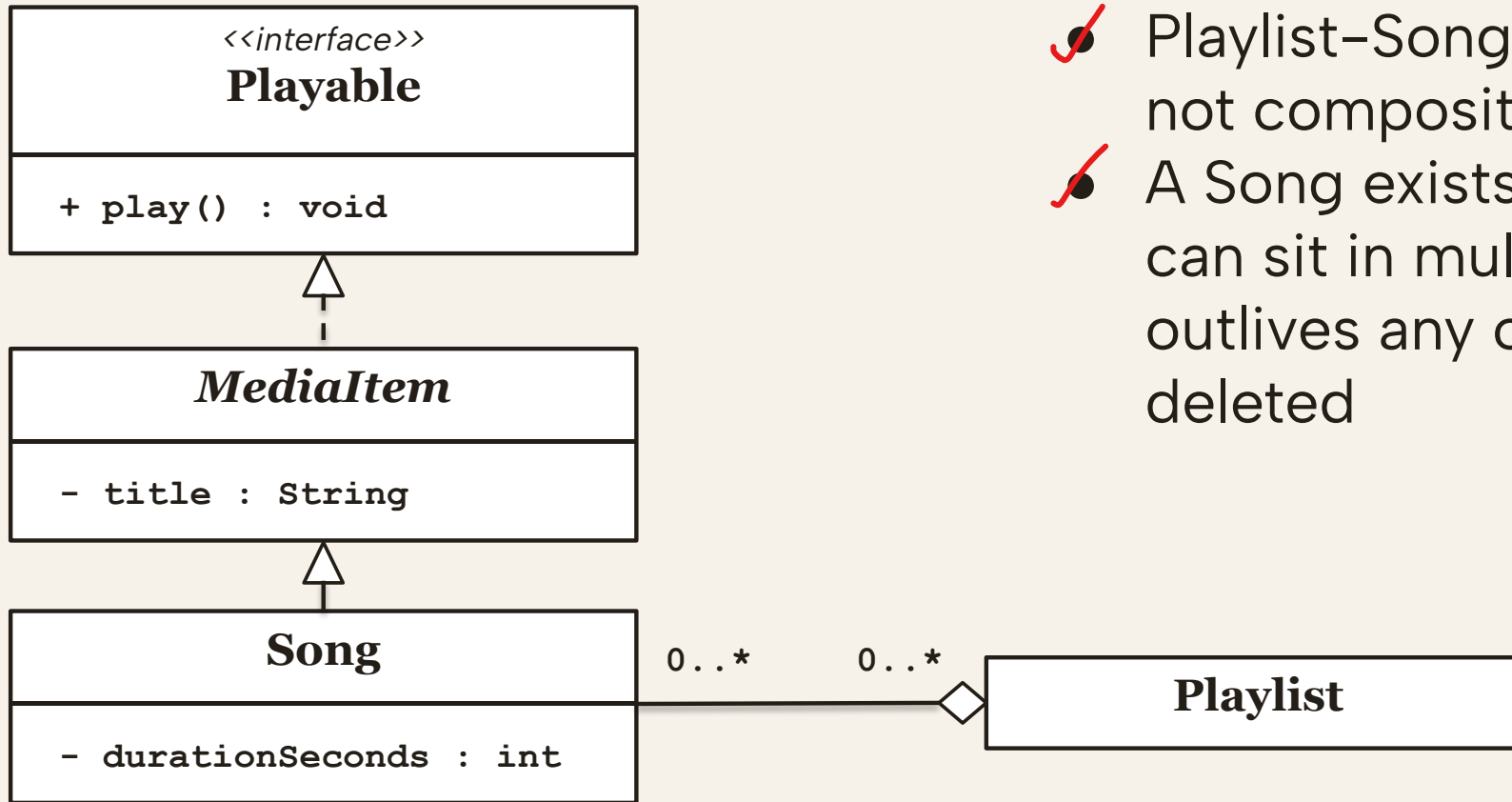
- Filled diamond → a Point created inside Shape (composition)
- Dashed triangle → implements; solid triangle → extends

✓ Practice Exercise: The Code

```
interface Playable {  
    void play();  
}  
abstract class MediaItem implements Playable {  
    private String title;  
}  
class Song extends MediaItem {  
    private int durationSeconds;  
}  
class Playlist {  
    private List<Song> songs;  
}
```

- ✓ ● Draw the class diagram
- ✓ ● Draw the relationship between Playlist and Song, and add multiplicity to both ends
- ✓ ● Decide: should Playlist-Song be aggregation or composition? Justify your choice

Solution



- ✓ Playlist–Song is aggregation, not composition
- ✓ A Song exists independently, can sit in multiple Playlists, and outlives any one Playlist being deleted

Thank You